PNY Trainings — Cyber Security Department

Web Application Security — Final Project Assignments

Project 2: Injection Hunter

Proof-of-Concept Write-Up: SQL Injection &
Cross-Site Scripting in OWASP Juice Shop

Student: Hamza Iqbal

Environment: Local lab — Docker on Windows 11 (own host)

Table of Contents

1. Objective

2. Scope & Rules of Engagement

3. Tools Used

4. Finding 1 — SQL Injection (Authentication Bypass)

5. Finding 2 — DOM-Based Cross-Site Scripting (XSS)

6. Findings Summary

7. Conclusion

1. Objective

The objective of this project was to find, exploit, and document one SQL injection (SQLi) and one cross-site scripting (XSS) vulnerability in the target application, and to produce developer-side fix recommendations for each. Both flaws were exploited with working proofs of concept (PoCs) in the local lab, captured as screenshot evidence, and mapped to concrete remediations.

2. Scope & Rules of Engagement

- **Target —** Target: OWASP Juice Shop v20.3.0, deployed locally via Docker, at http://localhost:3000
- **Environment —** Isolated lab on the tester's own Windows 11 host. Both attacks were executed only against this local instance.
- **Boundary —** No external system was touched, consistent with the lab-only scope of the assignment.

3. Tools Used

Tool	Version	Purpose
Docker Desktop	4.95.0	Container runtime for the target application
OWASP Juice Shop	20.3.0 (bkimminich/juice-shop)	Target application under test
Google Chrome	155	Manual exploitation and evidence capture

4. Finding 1 — SQL Injection (Authentication Bypass)

Severity: Critical

Affected endpoint: Login form at /#/login, processed by POST /rest/user/login

CWE: CWE-89 — Improper Neutralization of Special Elements used in an SQL Command

Summary. The login form builds its database query by concatenating the submitted email and password directly into an SQL statement. By closing the email string and appending an always-true condition, an attacker can neutralize the password check entirely and authenticate as the first account in the database — the administrator — without knowing any password.

Root cause

The backend interpolates user input into the query string instead of binding it as data. The resulting query is structurally equivalent to:

```
SELECT * FROM Users WHERE email = '<USER INPUT>' AND password = '<HASH>'
```

Any quote character in the input terminates the string literal and lets the attacker write SQL.

Proof of concept

Step 1: Open http://localhost:3000/#/login.

Step 2: In the Email field, enter the payload: ' or 1=1--

Step 3: In the Password field, enter any value (e.g. test).

Step 4: Click Log in.

Step 5: The application authenticates the session as the first user in the Users table — the administrator — and the "Login Admin" challenge is solved (Figure 1).

Why it works: the payload turns the query into

```
SELECT * FROM Users WHERE email = '' or 1=1--' AND password = '...'
```

1=1 is always true, so the WHERE clause matches a row; the -- sequence comments out the remainder of the statement, deleting the password condition. The first matching row returned is the administrator account (id 1).

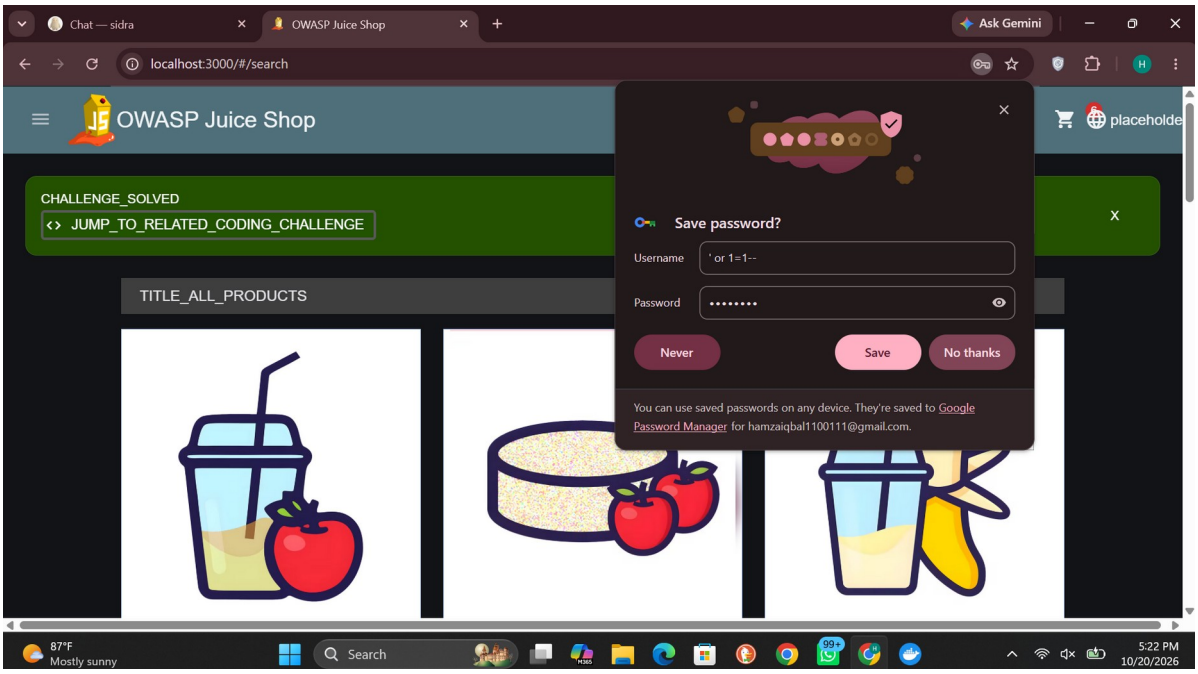


Figure 1 — SQLi login bypass: Chrome's save-password prompt shows the payload (' or 1=1--) submitted as the username, with the CHALLENGE_SOLVED banner confirming admin authentication

Impact

- Complete authentication bypass and administrator account takeover with no credentials.
- The same injection point can be extended (UNION-based or error-based extraction) to read the entire database, including password hashes and personal data.
- No user interaction and no prior access are required — the flaw is reachable by anyone who can open the login page.

Remediation

- **Primary fix —** Use parameterized queries / prepared statements everywhere. Bound parameters are sent to the database separately from the SQL text, so input can never change the query's structure.

```
// VULNERABLE — string concatenation
const q = `SELECT * FROM Users WHERE email = '${email}' AND password = '${hash}'`;

// FIXED — ORM / parameterized query (values bound as data, never as SQL code)
const user = await User.findOne({ where: { email: email } });
// or raw SQL with replacements:
// db.query('SELECT * FROM Users WHERE email = ?', { replacements: [email] })
```

- **Defense in depth —** Validate input server-side (e.g. email format allow-listing) as defense in depth — never as the primary control.
- **Hardening —** Run the application with a least-privilege database account, and log and alert on authentication anomalies.

5. Finding 2 — DOM-Based Cross-Site Scripting (XSS)

Severity: High

Affected functionality: Product search at /#/search?q=<input>

CWE: CWE-79 — Improper Neutralization of Input During Web Page Generation

Summary. The product search reflects the user's query into the results page and inserts it into the DOM without adequate sanitization. An attacker can inject an iframe whose javascript: source executes arbitrary JavaScript in the victim's browser, in the security context of the application.

Root cause

The search term travels from the URL fragment (q parameter) into the page's DOM. Plain <script> elements do not execute when inserted via innerHTML and are stripped by Angular's

sanitizer, but the application renders the search term through a path where an <iframe src="javascript:..."> survives — a classic sanitizer-bypass pattern. The browser then executes the script when the iframe loads.

Proof of concept

Step 1: Open http://localhost:3000 and click the search icon.

Step 2: In the search box, enter the payload: <iframe src="javascript:alert('xss')">

Step 3: Press Enter.

Step 4: The browser immediately executes the injected script — an alert box reading "xss" is displayed — and the "DOM XSS" challenge is solved (Figure 2). The payload is visible in the URL: /#/search?q=<iframe src="javascript:alert('xss')">.

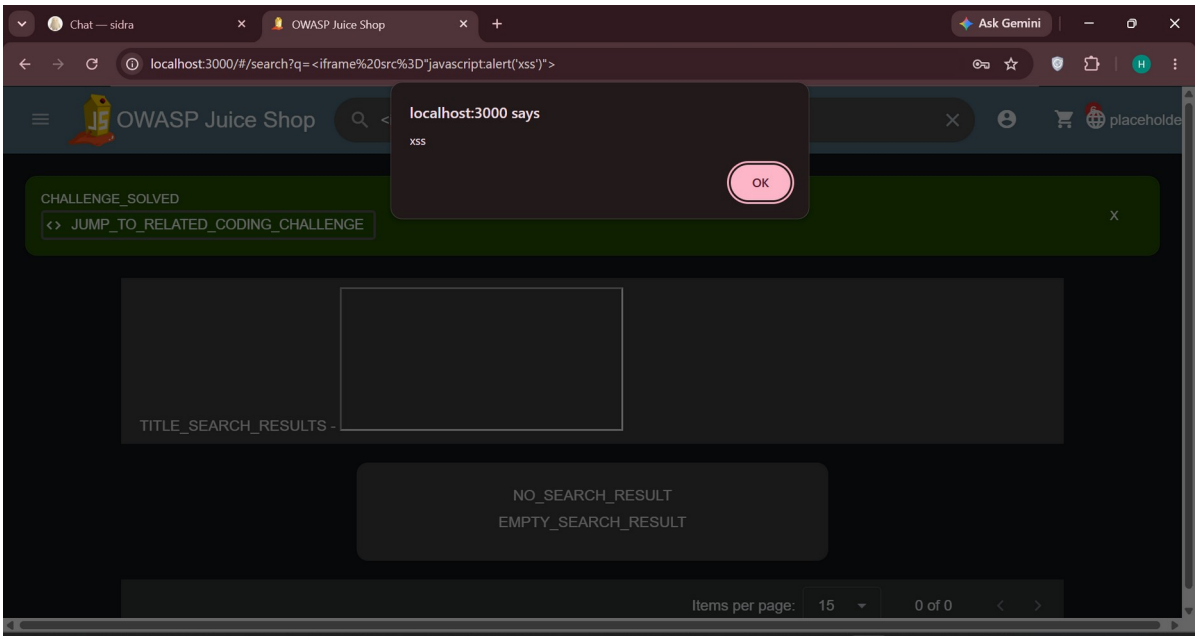


Figure 2 — DOM XSS: the injected iframe executes alert('xss'); the payload is visible in the address bar and the CHALLENGE_SOLVED banner confirms exploitation

Impact

- An attacker who tricks a user into opening a crafted search link runs script in that user's session — enabling session/token theft, credential phishing overlays, and actions performed as the victim.
- Because the payload lives in the URL fragment, it can be delivered by email or social media as an apparently legitimate link to the shop.

Remediation

- **Primary fix —** Never insert untrusted data into the DOM as HTML. Bind values as text and rely on the framework's default output encoding — Angular interpolation ({{ value }}) escapes content automatically.

```
// VULNERABLE — untrusted input rendered as HTML / sanitizer bypassed
element.innerHTML = searchTerm;
this.sanitizer.bypassSecurityTrustHtml(searchTerm);

// FIXED — bind as text; Angular encodes it
<span>{{ searchTerm }}</span>
// Never call bypassSecurityTrust* on user-controlled data.
```

- **Defense in depth —** Deploy a restrictive Content-Security-Policy header (e.g. script-src 'self') so injected script sources — including javascript: URIs — are blocked even if a sink is missed.
- **Hardening —** Validate and length-limit the search input server-side, and audit all DOM sinks (innerHTML, document.write, jQuery .html()) across the codebase.

6. Findings Summary

#	Vulnerability	Location	Severity	Status
1	SQL Injection — authentication bypass	/#/login → POST /rest/user/login	Critical	Exploited (admin session); fix documented
2	DOM-Based XSS	/#/search?q=	High	Exploited (script execution); fix documented

7. Conclusion

Project 2 is complete. Both target vulnerability classes were found and exploited with working proofs of concept: a Critical SQL injection that yields administrator access without credentials, and a High-severity DOM-based XSS that executes attacker script in a victim's browser. Each finding is supported by screenshot evidence and paired with a developer-side remediation whose common theme is a single principle: keep user input out of the structure of code — bind it as data in SQL, and render it as text in the DOM. The input points catalogued in Project 1 (search, login, feedback) proved to be exactly where the flaws lived, confirming the value of mapping the attack surface before testing it.